Traduzir

How Bridgewater Engineers a Research Ager

Five design decisions from PAT that do not need a 50-year archive, and the point time test the talk leaves open.



STEFAN JANSEN
JUL 29, 2026

7 1

Bridgewater's applied AI team published a talk on the Pocket Analyst Tool (PAT) research analyst deployed internally to hundreds of their investors for several mo Brendan McManus, Michael Ran, and Santi Weight spent 25 minutes on how it i built, with architecture diagrams and accuracy numbers. Most of what they desc not about models.

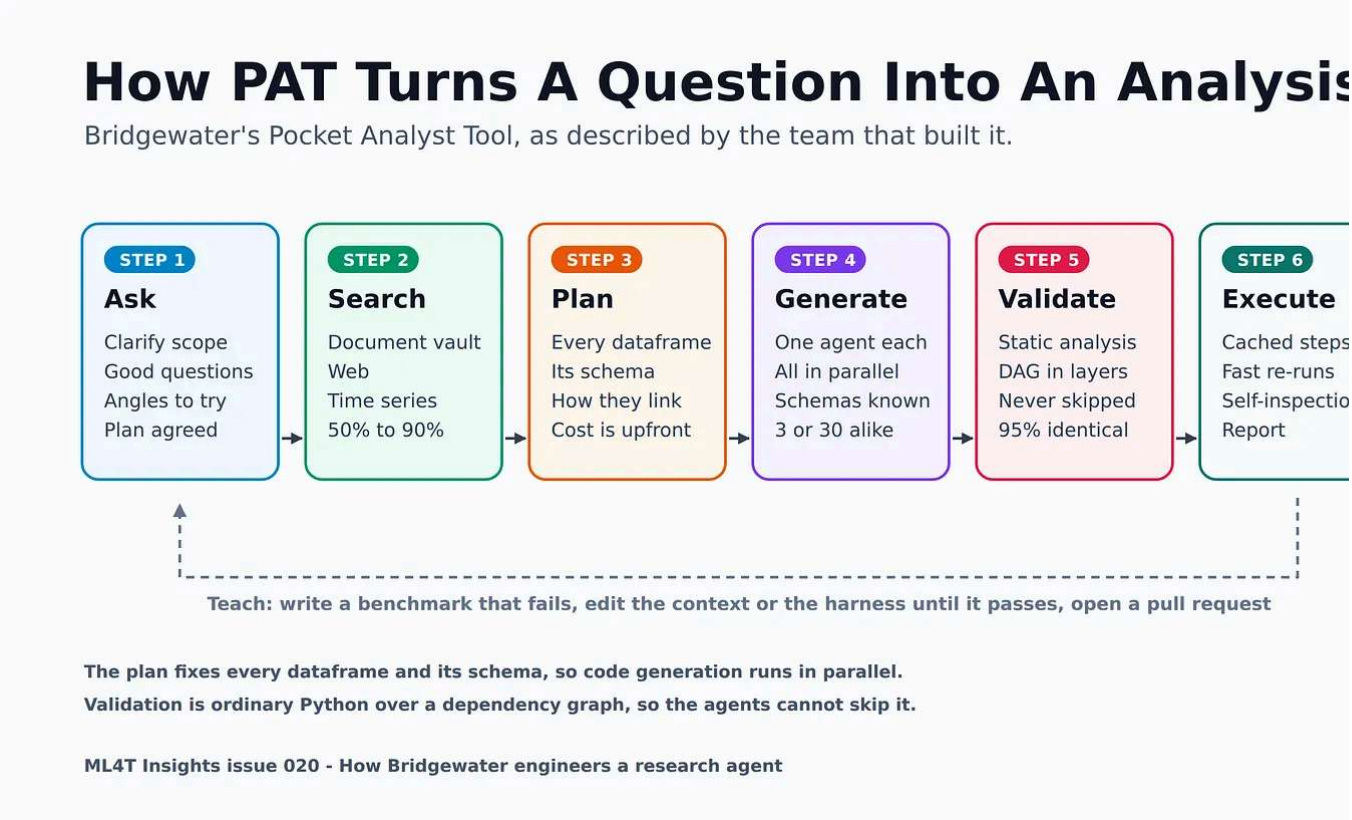
McManus sets the scope before the demo runs:

> PAT is not about how we trade. It's really about performing deep exploratory research, enabling our investors to go after questions that they never would ha had the bandwidth to pursue before.

Asked in the comments what the return has been, Weight replied that exact num are hard, then named time saved on analyses, "expected multiple man-years in fi months", and investors trying things they would otherwise have dropped, which marks as value-add rather than efficiency gain.

Their roadmap is a set of discrete sub-agents covering a research cycle: perceivir what is happening, forming questions, investigating, synthesizing, and folding w was learned back into shared memory. PAT is the investigation piece. Its outputs in the same database its inputs came from, so one analysis becomes an input to t next, and human and agent work accumulate in one place.

The AIA lab has also published the [AIA Forecaster](). On Saturday, August 1, I introduce engineering by demonstrating how to [design and build this multi-agent forecasting archi end-to-end for production deployment]().



PAT's pipeline: clarify, plan, generate, analyze dependencies, execute, validate, with a T loop feeding corrections back into context and harness

Domain inspection took retrieval from 50% to 90%

PAT searches a time series database holding tens of millions of series, some exte like the price of oil, some internal like their own 12-month-forward inflation esti The search agent starts with what you would expect. Ran:

> The search agents use traditional search techniques like RAG, re-ranking, et c but the thing we found to be a huge difference-maker was layering on an elem

> human-like inspection.

The inspection is what a researcher does without narrating it. You do not accept series because its name matched. You look at the frequency, you look at the curre and you check whether the values agree with what you already believe about the

> So embedding this sort of reasoning into our search agent is something that g up from roughly 50% accuracy all the way to 90%.

Fifty to ninety, from encoding domain judgment as search logic. Both numbers a Bridgewater's own; in the evaluation, the talk does not describe how many queri what counted as correct, or whether the two measurements share a test set. The direction is the transferable part. The reflex when retrieval underperforms is to r for a better embedding model or a reranker. Their gain came from writing down an analyst checks before trusting a series.

The dataframe plan is an intermediate representation

PAT resolves ambiguity with the user first, then plans. The planning phase name every dataframe the analysis will produce, the schema of each one, and how they connect. Ran calls the cost deliberate, and it buys something specific.

Because every task declares its output schema up front, code generation for all o runs in parallel. A visualization task at the end of the plan already knows the sha data produced by a loading task whose code does not exist yet.

> So this lets an analysis with three data frames or a medium-sized one with something like 30 data frames take roughly the same amount of time to gener code in both scenarios.

Their benchmark names its comparison. For the same context and the same plan generates code about four times faster than Claude Code, and a 20-task plan cost what a three-task plan costs.

The structure has a published analog. LLMCompiler (Kim et al., ICML 2024) plan directed acyclic graph of tasks with their dependencies and dispatches them to a parallel executor, reporting up to 3.7x lower latency and roughly 9% better accura than ReAct on its benchmarks. LLMCompiler's graph is over function calls. Bridgewater's is over dataframe schemas, which makes the plan a domain-specif intermediate representation: code generation lowers it into Python, and the unit compiled is an entire analysis. What the schema buys is that downstream work c start before upstream implementations exist.

Weight pushes the framing further. The plan is a natural language Python projec the to-do list a coding agent keeps, and two agents compiling the same task shou produce code that computes the same values.

Validation is mandatory, not discretionary

Weight's background is compiler theory. A compiler lowers user code to somethi executable, and it treats determinism and correctness as requirements rather tha aspirations. A coding agent lowers a plan to Python. He argues for building the s the way the industry already knows how to build the first.

Concretely: after generation, static analysis builds the dependency graph, and validation agents run in parallel across its layers. A five-task plan collapses to th layers, a twenty-task plan to four or five. The orchestration is ordinary Python.

> The main point I want you to take away from this part of the slides is that we enforce correctness in the architecture. Again, no agentic orchestration. This regular Python code, so the guardrails are really hard, and the agents cannot f to validate. They are forced to validate.

Their phrasing is worth reading narrowly, because the narrow version is the usef one. Ordinary Python control flow guarantees that every generated task passes through the prescribed validation stages, which removes the failure mode of an a deciding this particular output does not need checking. It guarantees nothing ab whether the validators test the right properties, whether the expected values are

or whether a task and its validator share a common-mode error. The architecture makes validation happen; whether the validation is sufficient is a question for th suite.

There is evidence behind putting the check outside the agent's discretion. Huang (ICLR 2024) tested intrinsic self-correction, where a model revises its own answe using nothing but its own judgment, and found that reasoning performance fails improve and, in their experiments, frequently degrades. They frame that setting one that matters in practice, since high-quality external feedback is often unavai PAT does inspect its own output, which Ran compares to a junior analyst double checking work before handing it over, and that inspection at least has computed and rendered charts to work against. The paper's finding applies to the case the architecture removes: judgment as the only check.

The result:

> The result is that when we run our test suite on any plan, 95% of the time, the that comes out is exactly the same for two different agents. So it's essentially a deterministic coding agent.

"Exactly the same" is doing unexamined work here. Identical source, identical computed values, and semantically equivalent code are three different claims, an talk names neither which one it measured nor how large the suite is. Read as hig reproducibility on their internal suite, it still buys something. Evaluation survive nondeterminism; it just costs more runs: when two runs of the same plan can dis attributing a change to your prompt means averaging over enough runs to separa effect from generation variance. Their phrase for the alternative is "vibes-based LLM-as-judge evals."

That aside is fairer to judges than it reads. Zheng et al. (2023), the work that popularized the method, report agreement with human preferences above 80% a document in the same paper that judges exhibit position bias, verbosity bias, anc enhancement bias, the last being a model's preference for its own output. Where

property can be checked by an executable test, the test is the more direct instrum
Open-ended qualities still need a judge, with that drift priced in.

Incremental execution on a runtime they own

PAT does not invoke its own generated code through a terminal. Bridgewater ru
for the model, injecting caching annotations by static analysis so intermediates a
never recomputed. Their second benchmark takes the last chart in a plan and ch
its name. Claude Code reruns the lot; PAT re-executes essentially instantly. Smal
to a finished analysis stop carrying the cost of a full iteration, so an investor can
adjusting one while still thinking about it.

This is available to Bridgewater because they own the execution environment. A
agent shelling out to a terminal cannot inject caching into a runtime it does not
control, which is the same point as treating the harness as context plus tools.

The transferable idea is incremental execution over an explicit dependency grap
it arrives with the problem every build system has. A renamed chart is the easy c
The graph tells you what to recompute only if the cache key captures code chang
data versions, parameters, and the relevant environment; get invalidation wrong
stale intermediates survive an edit that should have killed them.

Observed failures become regression benchmarks

A user who thinks PAT should have done better presses a Teach button, and an a
reads the conversation for behavioral mistakes, context gaps, or steering that cou
have been anticipated. Then, in order: it writes a benchmark **expected to fail**,
confirming the bad behavior reproduces. It edits the context repositories or the
harness until that benchmark passes. It verifies the rest of the suite still passes, s
fix cannot silently break an earlier one. It opens a pull request, which arrives in S
for a human.

This is test-driven development pointed at agent behavior. Writing the failing te is what ties the change to the behavior a user actually reported. Who defines the expected behavior decides whether the loop is worth anything, because a convers can reveal a user's preference as readily as an objective error. The human holds t step twice: the user can revise the proposed lesson before it becomes a benchma and a reviewer decides whether the pull request changes PAT for everyone. Agen also mine completed conversations for the same material without being asked, a correction reaches every investor at the firm rather than only the one who hit the problem.

One PAT per investor, differing by context and tools

Different investors at Bridgewater see different things. Someone with access to f positions needs a PAT that knows them; an analyst without that access needs a P that cannot leak them. Ran describes the resolution in one line:

> Practically speaking, this uniqueness is just a function of what context and wh tools each person's PAT has.

Every investor runs their own PAT, and the difference between any two is which context is loaded and which tools are exposed. That is a clean account of personalization, and it is not a security boundary. The data and tool layers still h enforce each investor's permissions, because a context-assembly rule that is too l or a tool that reaches further than intended, exposes data whether or not the mo was told to respect the limit.

What the team says it learned

Weight leaves the audience with three things: specialize agents instead of buildi generic, powerful ones; benchmark narrow workflows heavily and hill climb ther treat agentic coding as a compiler problem. The first comes with a warning.

> We don't really believe in generic, powerful agents. They make really cool den
> I'm sure many of us have given them. But it's really hard to make that a daily
> workflow that you can depend on.

What does not carry over

Bridgewater has been writing down the causal logic behind every trade since a 1
bond system on a yellow legal pad, and codifying it into machine-readable form
since. McManus is direct about the advantage:

> We didn't have to go back and write down everything for agents. It was alread
> there for us to draw upon.

Fifty years of codified investment logic, tens of millions of modeled series, and millions of documents arriving at thousands a day. Few teams can reproduce tha the talk says plainly that it is the foundation the rest sits on. The second advanta how the work is staffed. McManus describes multi-archetype teams with investo technologists and scientists sitting side by side: "Investors bring the context and domain-specific expertise. Technologists bring the architectural capability. And scientists bring the rigor."

Five things carry over without a hedge fund's archive: encoding domain inspecti into retrieval, planning in enough detail that generation parallelizes, enforcing validation structurally so it cannot be skipped, converting observed mistakes int failing benchmarks, and treating the harness as context plus tools. Each still nee schemas, metadata, and access rules of your own, which is real work. None of the needs fifty years of it.

The unanswered point-in-time test

The highest-rated comment on the talk proposes a test: reconstruct a past thesis "using only information available on that date," which would "separate fluent ret from genuine research discipline." That is point-in-time correctness, and it decic

whether a research system is measuring anything at all. Nothing in the talk says PAT handles it.

The same firm's AIA Labs has published on exactly that problem for a different system. The AIA Forecaster technical report treats foreknowledge bias, informat reaching the model from past its intended cutoff, as a first-class evaluation targe They built an LLM judge that flags search results implying knowledge beyond th cutoff, hand-audited 502 sampled traces to calibrate it, and estimate that about 1 of search results carry some leakage. Bounding the worst case moves their headli Brier score from 0.1159 to 0.1166. They also built MarketNightly, a live benchma drawing only on active prediction markets, where foreknowledge is impossible b construction rather than merely detected.

The forecasting results are worth stating precisely, because the direction change the benchmark. On ForecastBench, the AIA Forecaster is statistically indistinguishable from expert superforecasters, scoring 0.1076 against their 0.11 one variant and 0.1099 against 0.1152 on the other, where lower is better. On MarketLiquid, a harder benchmark sourced from liquid prediction markets, it tr the market consensus, 0.1258 against 0.1106; a regression ensemble of the two re 0.106 and beats both. Matching superforecasters on one benchmark and losing t liquid market on a harder one both appear in the same paper. That is the standar reporting to want before believing an agent result.

On Saturday, August 1, I teach this architecture by building it: a forecasting agent end-t working from my own implementation of the AIA Forecaster above. Prompt, context, ha loop. Typed tool contracts, agent state and replay, point-in-time research agents running parallel, a supervisor that reconciles their disagreement, and evaluation with proper scor 10:00 am to 3:30 pm ET, live. Details and registration.

Subscribe to ML for Trading Insights

By Stefan Jansen · Launched 4 months ago

Notes on ML for trading — and applied AI more broadly — from the author of the third editior

Subscribed

By subscribing, you agree Substack's Terms of Use, and acknowledge its Information Collection Notice and Privacy Policy.



7 Likes · 1 Restack

Previous Nex

Discussion about this post

Comments Restacks

Write a comment...

© 2026 Stefan Jansen · Privacy · Terms · Collection notice
Substack is the home for great culture